```
#include "sdcard.h"

/* ================================================================
  FUNÇÕES INTERNAS (LOW LEVEL)
  ================================================================ */
static SD_Status sd_find_file_entry(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file,
uint16_t *entry_out, uint32_t *lba_out);
/**
 * @brief Envia comando SD (CMD) e retorna R1
 */
static SD_Status sd_cmd_send(SD_Context *ctx, uint8_t cmd, uint32_t arg, uint8_t crc)
{
   uint8_t r1;
   uint16_t timeout = 10000;

   ctx->hal.spi_tx_rx(0xFF);
   ctx->hal.cs_low();

   ctx->hal.spi_tx_rx(0x40 | cmd);
   ctx->hal.spi_tx_rx((uint8_t)(arg >> 24));
   ctx->hal.spi_tx_rx((uint8_t)(arg >> 16));
   ctx->hal.spi_tx_rx((uint8_t)(arg >> 8));
   ctx->hal.spi_tx_rx((uint8_t)arg);
   ctx->hal.spi_tx_rx(crc);

   do {
      r1 = ctx->hal.spi_tx_rx(0xFF);
   } while (r1 == 0xFF && --timeout);

   if (timeout == 0) return SD_ERR_TIMEOUT;

   return (SD_Status)r1;
}

/**
 * @brief Aguarda o cartão SD não estar mais em estado busy (MISO != 0x00)
 *
 * Função interna com contador de ciclos baseado em F_CPU para evitar
 * travamento do firmware caso o cartão seja removido ou falhe eletricamente.
 * Timeout ≈ 250 ms em modo SPI_FAST (fosc/2).
 *
 * @param ctx Ponteiro para o contexto SD
 * @return SD_SUCCESS ou SD_ERR_TIMEOUT
 */
static SD_Status sd_spi_wait_ready(SD_Context *ctx)
{
   uint32_t timeout = F_CPU / 64UL;  /* ~250 ms @ 16 MHz em velocidade FAST */
   uint8_t data;

   do {
      data = ctx->hal.spi_tx_rx(0xFF);
      if (data != 0x00) {
```

```
        return SD_SUCCESS;
      }
    } while (--timeout);

    return SD_ERR_TIMEOUT;
}

/**
 * @brief Inicializa o cartão SD no modo SPI.
 * @param ctx Ponteiro para o contexto do cartão.
 * @return SD_SUCCESS em caso de sucesso, ou código de erro.
 */
SD_Status sd_init(SD_Context *ctx) {
    uint8_t i;
    SD_Status status = SD_ERR_TIMEOUT;

    // 1. Garante velocidade inicial segura (Lenta)
    if (ctx->hal.set_speed != NULL) {
        ctx->hal.set_speed(SD_SPI_SLOW);
    } else {
        sd_avr_spi_set_speed_default(SD_SPI_SLOW);
    }

    // Garante que o CS comece em nível alto
    ctx->hal.cs_high();

    // Sincronização inicial: 80 ciclos de clock
    for (i = 0; i < 10; i++) {
        ctx->hal.spi_tx_rx(0xFF);
    }

    // CMD0: Reset do cartão (Requer CRC 0x95)
    if (sd_cmd_send(ctx, 0, 0, 0x95) != 0x01) {
        return SD_ERR_INITIALIZATION;
    }

    // CMD8: Verifica voltagem (Requer CRC 0x87 e argumento 0x1AA)
    if (sd_cmd_send(ctx, 8, 0x1AA, 0x87) == 0x01) {
        // Lê os 4 bytes da resposta R7
        for (i = 0; i < 4; i++) ctx->hal.spi_tx_rx(0xFF);

        // ACMD41: Inicia processo de inicialização (CRC não é mais crítico aqui)
        for (uint16_t retry = 0; retry < 0xFFFF; retry++) {
            sd_cmd_send(ctx, 55, 0, 0xFF); // APP_CMD
            if (sd_cmd_send(ctx, 41, 0x40000000, 0xFF) == 0x00) {
                status = SD_SUCCESS;
                break;
            }
        }
    } else {
        return SD_ERR_VOLTAGE;
    }
```

```
  // 2. Se inicializou com sucesso, aumenta a velocidade do barramento
  if (status == SD_SUCCESS) {
    if (ctx->hal.set_speed != NULL) {
      ctx->hal.set_speed(SD_SPI_FAST);
    } else {
      sd_avr_spi_set_speed_default(SD_SPI_FAST);
    }

    // CMD16: Fixa tamanho do bloco em 512 bytes
    sd_cmd_send(ctx, 16, 512, 0xFF);
  }

  ctx->hal.cs_high();
  ctx->hal.spi_tx_rx(0xFF);

  return status;
}

SD_Status sd_init_retry(SD_Context *ctx)
{
  SD_Status s;

  for (uint8_t i = 0; i < 3; i++) {

    s = sd_init(ctx);

    if (s == SD_SUCCESS)
      return SD_SUCCESS;

    printf("Retry SD init (%u)\n", i+1);
    ctx->hal.delay_ms(100);
  }

  return s;
}

/* ================================================================
  STATUS
  ================================================================ */

uint16_t sd_get_status(SD_Context *ctx)
{
  uint8_t r1, r2;

  r1 = sd_cmd_send(ctx, 13, 0, 0xAF);
  r2 = ctx->hal.spi_tx_rx(0xFF);

  ctx->hal.cs_high();

  return ((uint16_t)r1 << 8) | r2;
}
```

```
/* ================================================================
  BLOCO - LEITURA
  ================================================================ */

SD_Status sd_blk_read(SD_Context *ctx, uint32_t block, uint8_t *buffer)
{
   uint32_t addr = (ctx->card_type == 1) ? (block << 9) : block;
   uint32_t timeout = 50000;
   uint8_t token;

   if (sd_cmd_send(ctx, 17, addr, 0xFF) != 0x00) {
      ctx->hal.cs_high();
      return SD_ERR_READY;
   }

   do {
      token = ctx->hal.spi_tx_rx(0xFF);
   } while (token != 0xFE && --timeout);

   if (timeout == 0) {
      ctx->hal.cs_high();
      return SD_ERR_TIMEOUT;
   }

   for (uint16_t i = 0; i < 512; i++)
      buffer[i] = ctx->hal.spi_tx_rx(0xFF);

   ctx->hal.spi_tx_rx(0xFF);
   ctx->hal.spi_tx_rx(0xFF);

   ctx->hal.cs_high();
   return SD_SUCCESS;
}

SD_Status sd_blk_read_multi(SD_Context *ctx, uint32_t start, uint16_t num, uint8_t *buffer)
{
   for (uint16_t i = 0; i < num; i++) {
      SD_Status s = sd_blk_read(ctx, start + i, &buffer[i * 512]);
      if (s != SD_SUCCESS) return s;
   }
   return SD_SUCCESS;
}

/**
 * @brief Escreve um bloco de 512 bytes
 *
 * @param ctx Ponteiro para o contexto
 * @param block Número do bloco
 * @param buffer Dados a serem escritos
 *
 * @return SD_Status
```

```
 */
SD_Status sd_blk_write(SD_Context *ctx, uint32_t block, const uint8_t *buffer)
{
   uint32_t addr = (ctx->card_type == 1) ? (block << 9) : block;

   /* ================================================================
     Verificação de status do cartão (CMD13) antes de escrita longa
     ================================================================ */
   if (sd_get_status(ctx) != 0x0000) {
      ctx->hal.cs_high();
      return SD_ERR_READY;
   }

   if (sd_cmd_send(ctx, 24, addr, 0xFF) != 0x00) {
      ctx->hal.cs_high();
      return SD_ERR_WRITE;
   }

   ctx->hal.spi_tx_rx(0xFE);

   for (uint16_t i = 0; i < 512; i++)
      ctx->hal.spi_tx_rx(buffer[i]);

   ctx->hal.spi_tx_rx(0xFF);
   ctx->hal.spi_tx_rx(0xFF);

   if ((ctx->hal.spi_tx_rx(0xFF) & 0x1F) != 0x05) {
      ctx->hal.cs_high();
      return SD_ERR_WRITE;
   }

   /* ================================================================
     Substitui while infinito por sd_spi_wait_ready (timeout padronizado)
     ================================================================ */
   if (sd_spi_wait_ready(ctx) != SD_SUCCESS) {
      ctx->hal.cs_high();
      return SD_ERR_TIMEOUT;
   }

   ctx->hal.cs_high();
   return SD_SUCCESS;
}

/**
 * @brief Escreve múltiplos blocos
 *
 * @param ctx Ponteiro para o contexto
 * @param start_block Bloco inicial
 * @param num_blocks Quantidade de blocos
 * @param buffer Dados
 *
 * @return SD_Status
```

```
 */
SD_Status sd_blk_write_multi(SD_Context *ctx, uint32_t start_block, uint16_t num_blocks, const
uint8_t *buffer)
{
   /* ================================================================
     Verificação de status do cartão (CMD13) antes de operação longa
     ================================================================ */
   if (sd_get_status(ctx) != 0x0000) {
      ctx->hal.cs_high();
      return SD_ERR_READY;
   }

   for (uint16_t i = 0; i < num_blocks; i++) {
      SD_Status s = sd_blk_write(ctx, start_block + i, &buffer[i * 512]);
      if (s != SD_SUCCESS) return s;
   }
   return SD_SUCCESS;
}

/* ================================================================
  FAT32
  ================================================================ */

/**
 * @brief Monta o sistema de arquivos FAT32, localizando a partição e parâmetros vitais.
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD.
 * @param fs  Ponteiro para a estrutura que armazenará os metadados do FAT32.
 * @return SD_Status Status da operação (SUCCESS, ERR_MBR, ERR_BOOT ou ERR_READY).
 */
SD_Status sd_fat_mount(SD_Context *ctx, FAT32_Info *fs)
{
   uint32_t partition_lba;
   uint32_t fat_size;
   uint32_t reserved_sectors;
   uint8_t num_fats;

   if (sd_blk_read(ctx, 0, ctx->buffer) != SD_SUCCESS)
      return SD_ERR_READY;

   if (ctx->buffer[510] != 0x55 || ctx->buffer[511] != 0xAA)
      return SD_ERR_MBR;

   partition_lba = (uint32_t)ctx->buffer[454] |
            ((uint32_t)ctx->buffer[455] << 8) |
            ((uint32_t)ctx->buffer[456] << 16) |
            ((uint32_t)ctx->buffer[457] << 24);

   if (sd_blk_read(ctx, partition_lba, ctx->buffer) != SD_SUCCESS)
      return SD_ERR_READY;

   if (ctx->buffer[510] != 0x55 || ctx->buffer[511] != 0xAA)
```

```
    return SD_ERR_BOOT;

  fs->sct_per_clus = ctx->buffer[13];

  reserved_sectors = (uint16_t)ctx->buffer[14] |
              ((uint16_t)ctx->buffer[15] << 8);

  num_fats = ctx->buffer[16];

  fat_size = (uint32_t)ctx->buffer[36] |
        ((uint32_t)ctx->buffer[37] << 8) |
        ((uint32_t)ctx->buffer[38] << 16) |
        ((uint32_t)ctx->buffer[39] << 24);

  fs->root_cluster = (uint32_t)ctx->buffer[44] |
              ((uint32_t)ctx->buffer[45] << 8) |
              ((uint32_t)ctx->buffer[46] << 16) |
              ((uint32_t)ctx->buffer[47] << 24);

  fs->fat_size = fat_size;

  fs->fat_lba = partition_lba + reserved_sectors;
  fs->data_start = fs->fat_lba + (num_fats * fat_size);
  fs->root_dir_lba = fs->data_start +
               (fs->root_cluster - 2) * fs->sct_per_clus;

  /* 🔥 NOVO: inicializa hint */
  fs->next_free_cluster = 2;

  return SD_SUCCESS;
}

/**
 * @brief Aloca um novo cluster e o encadeia ao final de uma sequência existente.
 *
 * @param ctx      Ponteiro para o contexto SD.
 * @param fs       Ponteiro para o sistema FAT32 montado.
 * @param last_cl   O cluster que era o fim da cadeia (será atualizado).
 * @param new_cl    Ponteiro para retornar o endereço do novo cluster alocado.
 * @return SD_Status Status da alocação e escrita na FAT.
 */
SD_Status sd_fat_chain_new_cluster(
  SD_Context *ctx,
  FAT32_Info *fs,
  uint32_t current_cluster,
  uint32_t *new_cluster
)
{
  /* ============================================================
    1. Aloca novo cluster
    ============================================================ */
  if (sd_fat_alloc_cluster(ctx, fs, new_cluster) != SD_SUCCESS)
```

```
        return SD_ERR_DISK_FULL;

    /* ============================================================
       2. Atualiza FAT[current] = new_cluster
       ============================================================ */
    uint32_t sector = fs->fat_lba + (current_cluster / 128);
    uint16_t offset = (current_cluster % 128) * 4;

    if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_READY;

    uint32_t entry = *((uint32_t *)&ctx->buffer[offset]);
    entry &= 0xF0000000;
    entry |= (*new_cluster & 0x0FFFFFFF);

    *((uint32_t *)&ctx->buffer[offset]) = entry;

    if (sd_blk_write(ctx, sector, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_WRITE;

    return SD_SUCCESS;
}

uint32_t sd_fat_next_cluster(SD_Context *ctx, uint32_t fat_lba, uint32_t cluster)
{
    uint32_t sector = fat_lba + (cluster / 128);
    uint16_t offset = (cluster % 128) * 4;

    if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS)
        return 0xFFFFFFFF;

    uint32_t next = (uint32_t)ctx->buffer[offset] |
            ((uint32_t)ctx->buffer[offset+1] << 8) |
            ((uint32_t)ctx->buffer[offset+2] << 16) |
            ((uint32_t)ctx->buffer[offset+3] << 24);

    return next & 0x0FFFFFFF;
}

/* ================================================================
   FILE
   ================================================================ */
/**
 * @brief Verifica se um arquivo existe no diretório raiz.
 *
 * @param ctx   Ponteiro para o contexto SD.
 * @param fs    Ponteiro para o sistema FAT32 montado.
 * @param file  Nome do arquivo (8.3).
 * @return SD_Status SUCCESS se encontrado, NOT_FOUND caso contrário.
 */
SD_Status sd_file_exists(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file)
{
```

```
    uint16_t entry_offset;
    uint32_t entry_lba;

    // Substitui o loop for fixo de 512 bytes pela busca dinâmica
    return sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba);
}

/**
 * @brief Obtém o tamanho de um arquivo em bytes.
 *
 * @param ctx   Ponteiro para o contexto SD.
 * @param fs    Ponteiro para o sistema FAT32 montado.
 * @param file  Nome do arquivo (8.3).
 * @param size  Ponteiro para armazenar o tamanho lido.
 * @return SD_Status Status da operação.
 */
SD_Status sd_file_get_size(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t *size)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    // Localiza a entrada em qualquer lugar da cadeia de clusters do root
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
        return SD_ERR_NOT_FOUND;
    }

    // O buffer já contém o setor correto graças à sd_find_file_entry
    *size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    return SD_SUCCESS;
}

/* ==================================================================
   UTIL
   ================================================================== */

void sd_spi_set_high_speed(SD_Context *ctx)
{
    (void)ctx; // depende do HAL externo
}

/**
 * @brief Lê o conteúdo de um arquivo FAT32 para um buffer externo
 *
 * Percorre a cadeia de clusters (cluster chain) e copia os dados
 * sequencialmente para o buffer fornecido.
 */
SD_Status sd_file_read(
    SD_Context *ctx,
    FAT32_Info *fs,
    const SD_FileName *file,
    uint8_t *buffer,
```

```
    uint32_t buffer_size
)
{
    uint32_t file_size = 0;
    uint32_t bytes_read = 0;

    /* ============================================================
       1. Ler diretório raiz
       ============================================================ */
    if (sd_blk_read(ctx, fs->root_dir_lba, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_READY;

    /* ============================================================
       2. Buscar arquivo
       ============================================================ */
    for (uint16_t i = 0; i < 512; i += 32) {

        if (ctx->buffer[i] == 0x00) break;

        if (!memcmp(&ctx->buffer[i], file->name, 8) &&
            !memcmp(&ctx->buffer[i+8], file->ext, 3)) {

            /* ====================================================
               3. Obter cluster inicial
               ==================================================== */
            uint32_t cluster =
                ((uint32_t)ctx->buffer[i+26]) |
                ((uint32_t)ctx->buffer[i+27] << 8) |
                ((uint32_t)ctx->buffer[i+20] << 16) |
                ((uint32_t)ctx->buffer[i+21] << 24);

            /* ====================================================
               4. Obter tamanho do arquivo
               ==================================================== */
            file_size =
                (uint32_t)ctx->buffer[i+28] |
                ((uint32_t)ctx->buffer[i+29] << 8) |
                ((uint32_t)ctx->buffer[i+30] << 16) |
                ((uint32_t)ctx->buffer[i+31] << 24);

            /* ====================================================
               5. Verificar tamanho do buffer
               ==================================================== */
            if (file_size > buffer_size)
                return SD_ERR_DISK_FULL;

            /* ====================================================
               6. Percorrer cluster chain
               ==================================================== */
            while (cluster < 0x0FFFFFF8) {

                uint32_t lba = fs->data_start +
```

```
                (cluster - 2) * fs->sct_per_clus;

        /* ================================================
          7. Ler setores do cluster
          ================================================ */
        for (uint8_t s = 0; s < fs->sct_per_clus; s++) {

          if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
            return SD_ERR_READY;

          /* ============================================
            8. Copiar para buffer externo
            ============================================ */
          for (uint16_t b = 0; b < 512; b++) {

            if (bytes_read >= file_size)
              return SD_SUCCESS;

            buffer[bytes_read++] = ctx->buffer[b];
          }
        }

        /* ================================================
          9. Próximo cluster
          ================================================ */
        cluster = sd_fat_next_cluster(ctx, fs->fat_lba, cluster);

        if (cluster == 0xFFFFFFFF)
          return SD_ERR_READY;
      }

      return SD_SUCCESS;
    }
  }
  return SD_ERR_NOT_FOUND;
}

/**
 * @brief Lê um arquivo FAT32 em modo streaming
 *
 * Esta função percorre toda a cadeia de clusters (cluster chain)
 * de um arquivo e fornece os dados ao usuário através de um callback,
 * evitando a necessidade de alocar grandes buffers em RAM.
 *
 * A cada setor (512 bytes), o callback é chamado com os dados lidos.
 * O último bloco pode conter menos de 512 bytes, dependendo do tamanho do arquivo.
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD
 * @param fs Ponteiro para a estrutura do sistema FAT32 montado
 * @param file Ponteiro para o nome do arquivo (formato 8.3)
 * @param callback Função callback chamada para processar os dados lidos
 *
```

```
 * @return
 * - SD_SUCCESS: leitura concluída com sucesso
 * - SD_ERR_NOT_FOUND: arquivo não encontrado no diretório
 * - SD_ERR_READ: erro durante leitura de bloco ou FAT
 */

SD_Status sd_file_read_stream(
   SD_Context *ctx,
   FAT32_Info *fs,
   const SD_FileName *file,
   sd_stream_callback_t callback
)
{
   uint32_t file_size = 0;
   uint32_t bytes_read = 0;

   if (sd_blk_read(ctx, fs->root_dir_lba, ctx->buffer) != SD_SUCCESS)
      return SD_ERR_READY;

   for (uint16_t i = 0; i < 512; i += 32) {

      if (ctx->buffer[i] == 0x00) break;

      if (!memcmp(&ctx->buffer[i], file->name, 8) &&
         !memcmp(&ctx->buffer[i+8], file->ext, 3)) {

         uint32_t cluster =
            ((uint32_t)ctx->buffer[i+26]) |
            ((uint32_t)ctx->buffer[i+27] << 8) |
            ((uint32_t)ctx->buffer[i+20] << 16) |
            ((uint32_t)ctx->buffer[i+21] << 24);

         file_size =
            (uint32_t)ctx->buffer[i+28] |
            ((uint32_t)ctx->buffer[i+29] << 8) |
            ((uint32_t)ctx->buffer[i+30] << 16) |
            ((uint32_t)ctx->buffer[i+31] << 24);

         if (file_size == 0) {
            return SD_SUCCESS;
         }

         while (cluster < 0x0FFFFFF8) {

            uint32_t lba = fs->data_start +
                     (cluster - 2) * fs->sct_per_clus;

            for (uint8_t s = 0; s < fs->sct_per_clus; s++) {

               if (bytes_read >= file_size)
                  return SD_SUCCESS;
```

```
            if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
              return SD_ERR_READY;

            uint32_t remaining = file_size - bytes_read;
            uint16_t to_send = (remaining > 512) ? 512 : remaining;

            if (callback)
              callback(ctx->buffer, to_send);

            bytes_read += to_send;
          }

          cluster = sd_fat_next_cluster(ctx, fs->fat_lba, cluster);

          if (cluster == 0xFFFFFFFF)
            return SD_ERR_READY;
        }

        return SD_SUCCESS;
      }
    }

    return SD_ERR_NOT_FOUND;
}

/**
 * @brief Escreve dados em um arquivo FAT32 existente (modo streaming com RMW e offset)
 *
 * Esta função sobrescreve o conteúdo de um arquivo existente utilizando
 * sua cadeia de clusters (cluster chain), aplicando Read-Modify-Write (RMW)
 * e controle de offset global para garantir escrita sequencial correta.
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD
 * @param fs Ponteiro para a estrutura do sistema FAT32 montado
 * @param file Ponteiro para o nome do arquivo (formato 8.3)
 * @param callback Função responsável por fornecer os dados a serem escritos
 *
 * @return
 * - SD_SUCCESS: escrita concluída com sucesso
 * - SD_ERR_NOT_FOUND: arquivo não encontrado
 * - SD_ERR_WRITE: erro durante leitura/escrita
 *
 * @note
 * - NÃO altera o tamanho do arquivo
 * - NÃO aloca novos clusters
 * - Escrita limitada ao tamanho existente do arquivo
 */
SD_Status sd_file_write_stream(
    SD_Context *ctx,
    FAT32_Info *fs,
    const SD_FileName *file,
    sd_write_callback_t callback
```

```
)
{
    uint32_t bytes_written = 0;
    uint32_t file_size = 0;

    /* Buffer temporário para dados do usuário */
    uint8_t temp[512];

    /* ================================================================
       Verificação de status do cartão (CMD13) antes de operação longa
       ================================================================ */
    if (sd_get_status(ctx) != 0x0000) {
        ctx->hal.cs_high();
        return SD_ERR_READY;
    }

    /* ============================================================
       1. Ler diretório raiz
       ============================================================ */
    if (sd_blk_read(ctx, fs->root_dir_lba, ctx->buffer) != SD_SUCCESS)
        return SD_ERR_READY;

    /* ============================================================
       2. Buscar arquivo
       ============================================================ */
    for (uint16_t i = 0; i < 512; i += 32) {
        if (ctx->buffer[i] == 0x00) break;

        if (!memcmp(&ctx->buffer[i], file->name, 8) &&
            !memcmp(&ctx->buffer[i+8], file->ext, 3)) {

            /* Obter cluster inicial e tamanho */
            uint32_t cluster =
                ((uint32_t)ctx->buffer[i+26]) |
                ((uint32_t)ctx->buffer[i+27] << 8) |
                ((uint32_t)ctx->buffer[i+20] << 16) |
                ((uint32_t)ctx->buffer[i+21] << 24);

            file_size =
                (uint32_t)ctx->buffer[i+28] |
                ((uint32_t)ctx->buffer[i+29] << 8) |
                ((uint32_t)ctx->buffer[i+30] << 16) |
                ((uint32_t)ctx->buffer[i+31] << 24);

            /* Percorrer cluster chain */
            while (cluster < 0x0FFFFFF8) {
                uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus;

                for (uint8_t s = 0; s < fs->sct_per_clus; s++) {
                    if (bytes_written >= file_size)
                        return SD_SUCCESS;
```

```
            /* Ler setor atual (RMW) */
            if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
              return SD_ERR_WRITE;

            /* Obter dados do callback */
            uint16_t len = callback(temp, 512);
            if (len == 0) {
              /* Truncamento real + atualizar tamanho */
              uint16_t sector_offset = bytes_written % 512;
              if (sector_offset != 0) {
                if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                  return SD_ERR_WRITE;
                memset(&ctx->buffer[sector_offset], 0, 512 - sector_offset);
                if (sd_blk_write(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                  return SD_ERR_WRITE;
              }
              if (sd_file_update_size(ctx, fs, file, bytes_written) != SD_SUCCESS)
                return SD_ERR_WRITE;
              return SD_SUCCESS;
            }

            /* Limitar ao espaço restante */
            uint32_t remaining_file = file_size - bytes_written;
            if (len > remaining_file) len = remaining_file;

            uint16_t sector_offset = bytes_written % 512;
            uint16_t space_in_sector = 512 - sector_offset;
            uint16_t to_copy = (len < space_in_sector) ? len : space_in_sector;

            memcpy(&ctx->buffer[sector_offset], temp, to_copy);

            if (sd_blk_write(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
              return SD_ERR_WRITE;

            bytes_written += to_copy;
          }

          cluster = sd_fat_next_cluster(ctx, fs->fat_lba, cluster);
          if (cluster == 0xFFFFFFFF)
            return SD_ERR_READY;
        }
        return SD_SUCCESS;
      }
    }
    return SD_ERR_NOT_FOUND;
}

/**
 * @brief Atualiza o tamanho de um arquivo na sua entrada de diretório física.
 *
 * @param ctx      Ponteiro para o contexto SD.
 * @param fs       Ponteiro para o sistema FAT32 montado.
```

```
 * @param file     Nome do arquivo (8.3).
 * @param new_size  Novo tamanho em bytes.
 * @return SD_Status Status da gravação no disco.
 */
SD_Status sd_file_update_size(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t
new_size)
{
   uint16_t entry_offset;
   uint32_t entry_lba;

   // 1. Localiza a posição física da entrada
   if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
      return SD_ERR_NOT_FOUND;
   }

   // 2. Atualiza o tamanho no buffer (o buffer foi preenchido pela sd_find_file_entry)
   *((uint32_t *)&ctx->buffer[entry_offset + 28]) = new_size;

   // 3. Grava o setor de volta no LBA correto (não mais fixo no root_dir_lba)
   if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) {
      return SD_ERR_WRITE;
   }

   return SD_SUCCESS;
}

/**
 * @brief Adiciona dados ao final de um arquivo FAT32 (append streaming)
 *
 * Versão robusta com:
 * - Continuidade dentro do setor (RMW)
 * - Alocação dinâmica de clusters
 * - Sync incremental (power-loss safe)
 *
 * @param ctx Contexto SD
 * @param fs Sistema FAT32 montado
 * @param file Nome do arquivo (8.3)
 * @param callback Função que fornece os dados
 *
 * @return SD_Status
 */
SD_Status sd_file_append_stream(
   SD_Context *ctx,
   FAT32_Info *fs,
   const SD_FileName *file,
   sd_write_callback_t callback
)
{
   uint32_t file_size, cluster;
   uint16_t entry_off;
   uint32_t entry_lba;
   uint8_t data_buf[64]; // pequeno buffer para streaming
```

```
/* ==================================================================
  Verificação de status do cartão
  ================================================================== */
if (sd_get_status(ctx) != 0x0000) {
   ctx->hal.cs_high();
   return SD_ERR_READY;
}

/* ==================================================================
  Localiza arquivo
  ================================================================== */
if (sd_find_file_entry(ctx, fs, file, &entry_off, &entry_lba) != SD_SUCCESS)
   return SD_ERR_NOT_FOUND;

/* ==================================================================
  Obtém cluster inicial e tamanho
  ================================================================== */
cluster =
   ((uint32_t)ctx->buffer[entry_off + 26]) |
   ((uint32_t)ctx->buffer[entry_off + 27] << 8) |
   ((uint32_t)ctx->buffer[entry_off + 20] << 16) |
   ((uint32_t)ctx->buffer[entry_off + 21] << 24);

file_size = *((uint32_t *)&ctx->buffer[entry_off + 28]);

/* ==================================================================
  Navegar até o último cluster da cadeia
  ================================================================== */
uint32_t current_cl = cluster;
uint32_t next_cl;

while ((next_cl = sd_fat_next_cluster(ctx, fs->fat_lba, current_cl)) < 0x0FFFFFF8 && next_cl >= 2) {
   current_cl = next_cl;
}

uint32_t current_offset = file_size;
uint16_t bytes_to_write;

/* ==================================================================
  Loop principal de escrita
  ================================================================== */
while ((bytes_to_write = callback(data_buf, sizeof(data_buf))) > 0) {

   uint16_t pos = 0;

   while (pos < bytes_to_write) {

      uint32_t lba = fs->data_start +
                (current_cl - 2) * fs->sct_per_clus +
                ((current_offset / 512) % fs->sct_per_clus);
```

```
    uint16_t off = current_offset % 512;

    /* ==========================================================
      RMW se necessário
      ========================================================== */
    if (off != 0 || (bytes_to_write - pos) < 512) {
       if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS)
          return SD_ERR_READY;
    }

    uint16_t chunk = (512 - off < bytes_to_write - pos) ?
                (512 - off) :
                (bytes_to_write - pos);

    memcpy(&ctx->buffer[off], &data_buf[pos], chunk);

    if (sd_blk_write(ctx, lba, ctx->buffer) != SD_SUCCESS)
       return SD_ERR_WRITE;

    current_offset += chunk;
    pos += chunk;

    /* ==========================================================
      Avanço de cluster (se necessário)
      ========================================================== */
    if ((current_offset % 512) == 0) {

       if (sd_fat_next_cluster(ctx, fs->fat_lba, current_cl) >= 0x0FFFFFF8) {

          uint32_t new_cl;

          if (sd_fat_chain_new_cluster(ctx, fs, current_cl, &new_cl) != SD_SUCCESS)
             return SD_ERR_DISK_FULL;

          current_cl = new_cl;
       }

       /* ======================================================
         🔥 NOVO: Sync incremental (segurança contra falha)
         ====================================================== */
       if (sd_file_sync(ctx, fs, file, current_offset) != SD_SUCCESS)
          return SD_ERR_WRITE;
    }
  }
}

/* ================================================================
  Sync final
  ================================================================ */
if (sd_file_sync(ctx, fs, file, current_offset) != SD_SUCCESS)
   return SD_ERR_WRITE;
```

```
    return SD_SUCCESS;
}

SD_Status sd_file_create(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file) {
    uint32_t first_cl;
    uint16_t entry_off;
    uint32_t entry_lba;

    if (sd_find_file_entry(ctx, fs, file, &entry_off, &entry_lba) == SD_SUCCESS) return SD_SUCCESS;
    if (sd_fat_alloc_cluster(ctx, fs, &first_cl) != SD_SUCCESS) return SD_ERR_DISK_FULL;
    if (sd_blk_read(ctx, fs->root_dir_lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

    for (uint16_t i = 0; i < 512; i += 32) {
        if (ctx->buffer[i] == 0x00 || ctx->buffer[i] == 0xE5) {
            memset(&ctx->buffer[i], 0, 32);
            memcpy(&ctx->buffer[i], file->name, 8);
            memcpy(&ctx->buffer[i+8], file->ext, 3);
            ctx->buffer[i+11] = 0x20; // Archive
            ctx->buffer[i+20] = (uint8_t)(first_cl >> 16);
            ctx->buffer[i+21] = (uint8_t)(first_cl >> 24);
            ctx->buffer[i+26] = (uint8_t)first_cl;
            ctx->buffer[i+27] = (uint8_t)(first_cl >> 8);
            *((uint32_t *)&ctx->buffer[i+28]) = 0; // Size 0
            return sd_blk_write(ctx, fs->root_dir_lba, ctx->buffer);
        }
    }
    return SD_ERR_DISK_FULL;
}

SD_Status sd_fat_alloc_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t *new_cluster)
{
    uint32_t total_clusters = fs->fat_size * 128;
    uint32_t start = fs->next_free_cluster;
    uint32_t cluster = start;

    do {
        uint32_t sector = fs->fat_lba + (cluster / 128);
        uint16_t offset = (cluster % 128) * 4;

        if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS)
            return SD_ERR_READY;

        uint32_t entry = *((uint32_t *)&ctx->buffer[offset]);
        uint32_t val = entry & 0x0FFFFFFF;

        if (val == 0) {

            /* 🔥 Preserva bits altos */
            entry &= 0xF0000000;
            entry |= 0x0FFFFFFF;

            *((uint32_t *)&ctx->buffer[offset]) = entry;
```

```
        if (sd_blk_write(ctx, sector, ctx->buffer) != SD_SUCCESS)
          return SD_ERR_WRITE;

        *new_cluster = cluster;

        /* 🔥 Avança corretamente */
        fs->next_free_cluster = cluster + 1;

        return SD_SUCCESS;
      }

      cluster++;

      if (cluster >= total_clusters)
        cluster = 2;

    } while (cluster != start);

    return SD_ERR_DISK_FULL;
}

/**
 * @brief Localiza a entrada de um arquivo percorrendo toda a cadeia de clusters do diretório.
 *
 * @param ctx       Ponteiro para o contexto SD.
 * @param fs        Ponteiro para o FAT32 montado.
 * @param file      Nome do arquivo a buscar.
 * @param entry_out  Ponteiro para armazenar o índice (offset 0-511) dentro do buffer.
 * @param lba_out    Ponteiro para armazenar o LBA do setor onde a entrada foi achada.
 * @return SD_Status SUCCESS se achou, NOT_FOUND se percorreu tudo e não existe.
 */
static SD_Status sd_find_file_entry(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file,
uint16_t *entry_out, uint32_t *lba_out) {
    uint32_t cluster = fs->root_cluster;
    while (cluster < 0x0FFFFFF8 && cluster >= 2) {
      for (uint8_t s = 0; s < fs->sct_per_clus; s++) {
        uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus + s;
        if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
        for (uint16_t i = 0; i < 512; i += 32) {
          if (ctx->buffer[i] == 0x00) return SD_ERR_NOT_FOUND;
          if (ctx->buffer[i] == 0xE5) continue;
          if (!memcmp(&ctx->buffer[i], file->name, 8) && !memcmp(&ctx->buffer[i+8], file->ext, 3)) {
            *entry_out = i;
            *lba_out = lba;
            return SD_SUCCESS;
          }
        }
      }
      cluster = sd_fat_next_cluster(ctx, fs->fat_lba, cluster);
    }
    return SD_ERR_NOT_FOUND;
```

```
}

/**
 * @brief Percorre a tabela FAT e marca todos os clusters de uma cadeia como livres (0).
 *
 * @param ctx    Ponteiro para o contexto SD.
 * @param fs     Ponteiro para o FAT32 montado.
 * @param cluster O primeiro cluster da cadeia a ser liberada.
 * @return SD_Status Status da operação.
 */
static SD_Status sd_fat_free_chain(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster) {
  uint32_t next_cl;
  uint32_t fat_sec, last_sec = 0xFFFFFFFF;
  uint16_t offset;

  while (cluster >= 2 && cluster < 0x0FFFFFF8) {
    // 1. Obter o próximo cluster antes de apagar o atual
    next_cl = sd_fat_next_cluster(ctx, fs->fat_lba, cluster);

    // 2. Calcular posição do cluster atual na FAT
    fat_sec = fs->fat_lba + (cluster / 128); // 128 entradas de 4 bytes por setor
    offset = (cluster % 128) * 4;

    // 3. Otimização: Só lê o setor da FAT se ele for diferente do anterior
    if (fat_sec != last_sec) {
      if (sd_blk_read(ctx, fat_sec, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
      last_sec = fat_sec;
    }

    // 4. Marcar cluster como livre (0x00000000)
    *((uint32_t *)&ctx->buffer[offset]) = 0x00000000;

    // 5. Se o próximo cluster estiver em outro setor ou for o fim, grava o atual
    uint32_t next_fat_sec = fs->fat_lba + (next_cl / 128);
    if (next_fat_sec != fat_sec || next_cl >= 0x0FFFFFF8) {
      if (sd_blk_write(ctx, fat_sec, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;
    }

    cluster = next_cl;
  }
  return SD_SUCCESS;
}

/**
 * @brief Exclui um arquivo do diretório e libera seu espaço no disco.
 *
 * @param ctx  Ponteiro para o contexto SD.
 * @param fs   Ponteiro para o FAT32 montado.
 * @param file Nome do arquivo (8.3).
 * @return SD_Status Status da exclusão.
 */
SD_Status sd_file_delete(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file) {
```

```
    uint16_t entry_off;
    uint32_t entry_lba;
    uint32_t first_cluster;

    // 1. Localizar a entrada do arquivo
    if (sd_find_file_entry(ctx, fs, file, &entry_off, &entry_lba) != SD_SUCCESS) {
        return SD_ERR_NOT_FOUND;
    }

    // 2. Capturar o primeiro cluster (buffer já está carregado pelo find_file_entry)
    first_cluster = ((uint32_t)ctx->buffer[entry_off + 26]) |
                    ((uint32_t)ctx->buffer[entry_off + 27] << 8) |
                    ((uint32_t)ctx->buffer[entry_off + 20] << 16) |
                    ((uint32_t)ctx->buffer[entry_off + 21] << 24);

    // 3. Marcar entrada no diretório como deletada (0xE5)
    ctx->buffer[entry_off] = 0xE5;
    if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;

    // 4. Liberar a cadeia de clusters na FAT
    return sd_fat_free_chain(ctx, fs, first_cluster);
}

/**
 * @brief Converte uma string (ex: "teste.txt") para o formato 8.3 (ex: "TESTE   TXT").
 *
 * @param file_out Ponteiro para a estrutura SD_FileName que será preenchida.
 * @param name_in  String com o nome do arquivo (ex: "data.csv").
 * @return SD_Status SUCCESS ou ERR se o nome for inválido/longo.
 */
SD_Status sd_set_filename(SD_FileName *file_out, const char *name_in) {
    if (!file_out || !name_in) return SD_ERR_READY;

    // 1. Limpar a estrutura com espaços (padrão FAT)
    memset(file_out->name, ' ', 8);
    memset(file_out->ext, ' ', 3);

    uint8_t i = 0;
    // 2. Copiar o nome (até achar o ponto ou fim da string)
    while (name_in[i] != '.' && name_in[i] != '\0' && i < 8) {
        char c = name_in[i];
        if (c >= 'a' && c <= 'z') c -= 32; // Converter para maiúscula
        file_out->name[i] = c;
        i++;
    }

    // 3. Pular o ponto (se existir) e ir para a extensão
    const char *dot = strchr(name_in, '.');
    if (dot) {
        dot++; // Avança após o '.'
        for (uint8_t j = 0; j < 3 && dot[j] != '\0'; j++) {
            char c = dot[j];
```

```
        if (c >= 'a' && c <= 'z') c -= 32; // Converter para maiúscula
        file_out->ext[j] = c;
      }
    }

    return SD_SUCCESS;
}

/**
 * @brief Atualiza o setor FSInfo do FAT32 (Normalmente LBA 1).
 * Mantém o Windows feliz e evita mensagens de "unidade com erro".
 */
SD_Status sd_fat_update_fsinfo(SD_Context *ctx, FAT32_Info *fs, uint32_t free_clusters, uint32_t
last_alloc) {
    // 1. Ler o setor FSInfo (Geralmente partition_lba + 1)
    // No seu caso, você pode armazenar o fsinfo_lba no FAT32_Info durante o mount
    uint32_t fsinfo_lba = fs->fat_lba - (fs->fat_lba - fs->root_dir_lba > 32 ? 32 : 1); // Simplificado

    if (sd_blk_read(ctx, fsinfo_lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

    // 2. Verificar assinaturas de segurança do FSInfo (0x41615252 e 0x61417272)
    if (ctx->buffer[0] != 0x52 || ctx->buffer[1] != 0x52) return SD_ERR_BOOT;

    // 3. Atualizar campos (Offsets 488 e 492)
    *((uint32_t *)&ctx->buffer[488]) = free_clusters; // Clusters livres (0xFFFFFFFF = desconhecido)
    *((uint32_t *)&ctx->buffer[492]) = last_alloc;    // Último alocado

    return sd_blk_write(ctx, fsinfo_lba, ctx->buffer);
}

/**
 * @brief Configura os registradores SPI do AVR para modo lento ou rápido.
 * @note Compatível com a maioria dos microcontroladores AVR (ATmega328P, ATmega2560, etc).
 * Dispositivos com mapeamento de registradores não padrão (ex: XMEGA) podem exigir
 * que o desenvolvedor forneça sua própria função set_speed na struct HAL.
 *
 * @param speed_mode SD_SPI_SLOW (fosc/64) ou SD_SPI_FAST (fosc/2)
 */
void sd_avr_spi_set_speed_default(uint8_t speed_mode) {
    if (speed_mode == SD_SPI_SLOW) {
        SPCR = (1<<SPE)|(1<<MSTR)|(1<<SPR1)|(1<<SPR0);
    } else {
        SPCR = (1<<SPE)|(1<<MSTR);
        SPSR |= (1<<SPI2X);
    }
}

/**
 * @brief Sincroniza um arquivo FAT32 no disco (flush)
 *
 * Atualiza o tamanho do arquivo na entrada de diretório,
 * garantindo consistência em caso de queda de energia.
```

```
 *
 * NÃO altera clusters nem força escrita de buffer externo.
 *
 * @param ctx Contexto SD
 * @param fs Sistema FAT32 montado
 * @param file Nome do arquivo (8.3)
 * @param current_size Tamanho atual do arquivo em bytes
 *
 * @return SD_Status
 */
SD_Status sd_file_sync(
    SD_Context *ctx,
    FAT32_Info *fs,
    const SD_FileName *file,
    uint32_t current_size
)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    /* ============================================================
       1. Localiza entrada do arquivo
       ============================================================ */
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
        return SD_ERR_NOT_FOUND;
    }

    /* ============================================================
       2. Atualiza tamanho no buffer
       ============================================================ */
    *((uint32_t *)&ctx->buffer[entry_offset + 28]) = current_size;

    /* ============================================================
       3. Grava setor no SD
       ============================================================ */
    if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_WRITE;
    }

    return SD_SUCCESS;
}
```